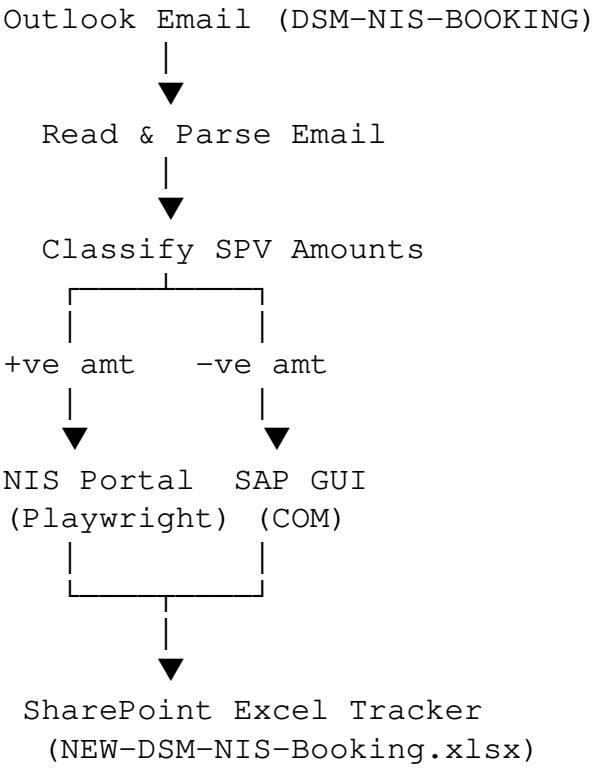
DSM-NIS Booking Automation Pipeline

An end-to-end automation pipeline for **AGEL (Adani Green Energy Limited)** that monitors a dedicated Outlook mailbox for DSM (Deviation Settlement Mechanism) emails, classifies the SPV financial fields, and automatically routes each transaction to either the **NIS portal** (positive amounts) or **SAP GUI** (negative amounts), then writes the result back to a SharePoint Excel tracker.

Table of Contents

- Overview
- Architecture
- Project Structure
- Pipeline Steps
- Data Flow
- Module Reference
- Configuration
- Environment Variables
- Installation
- Running the Pipeline
- Resume / Fault Tolerance
- SharePoint Excel Sheet Layout

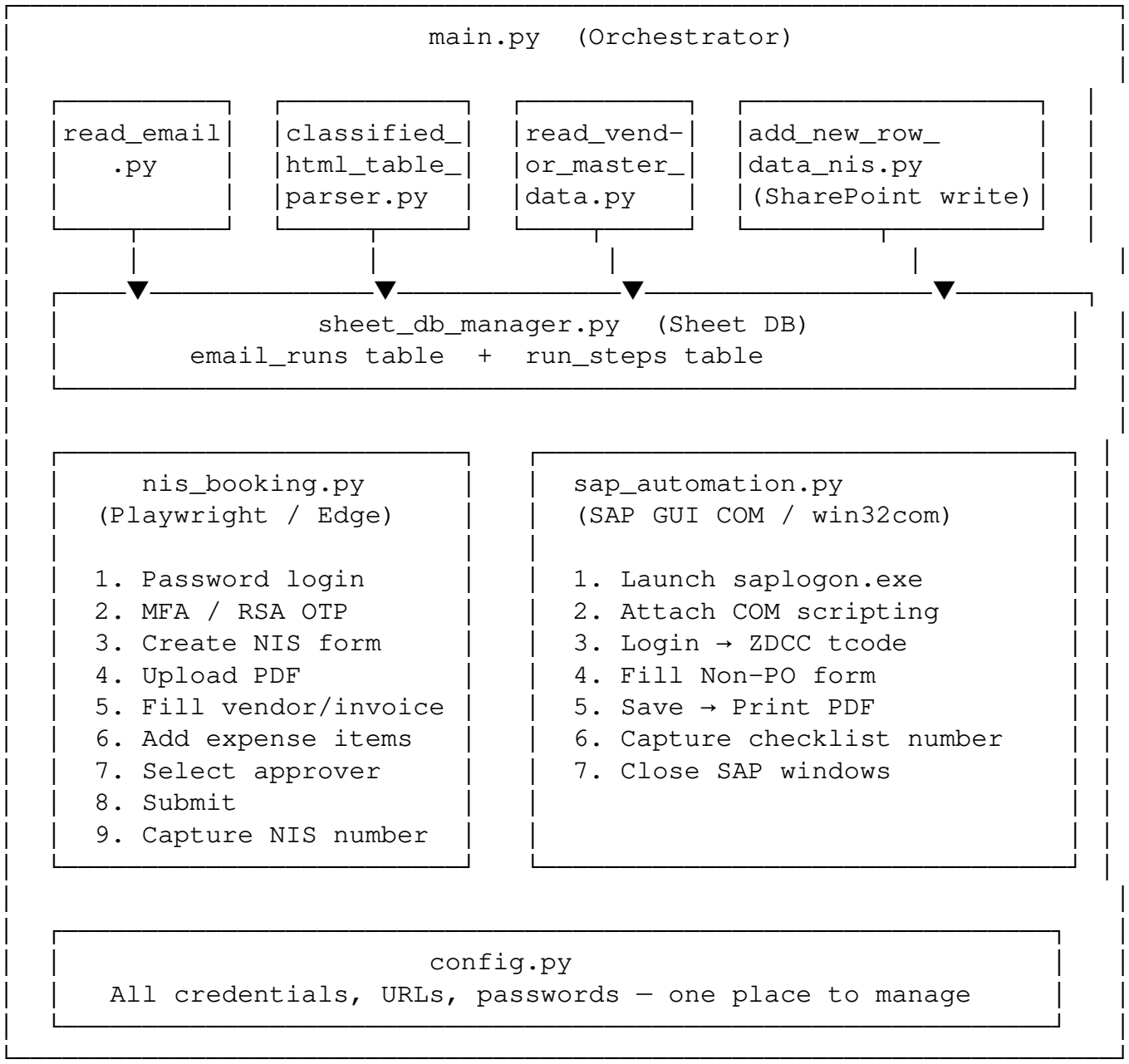
Overview



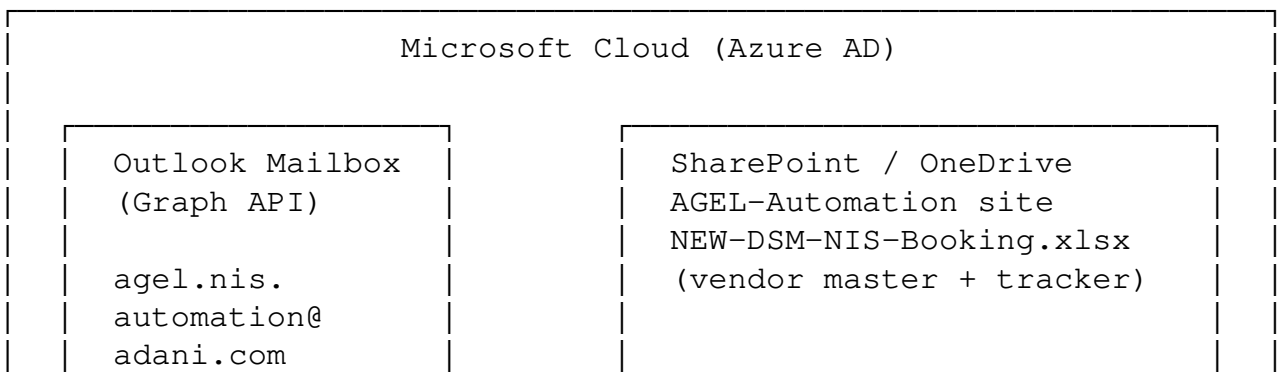
The pipeline is **resume-aware**: every step is recorded in a local Sheet DB workbook. If the process crashes or is interrupted, restarting it automatically picks up from the first

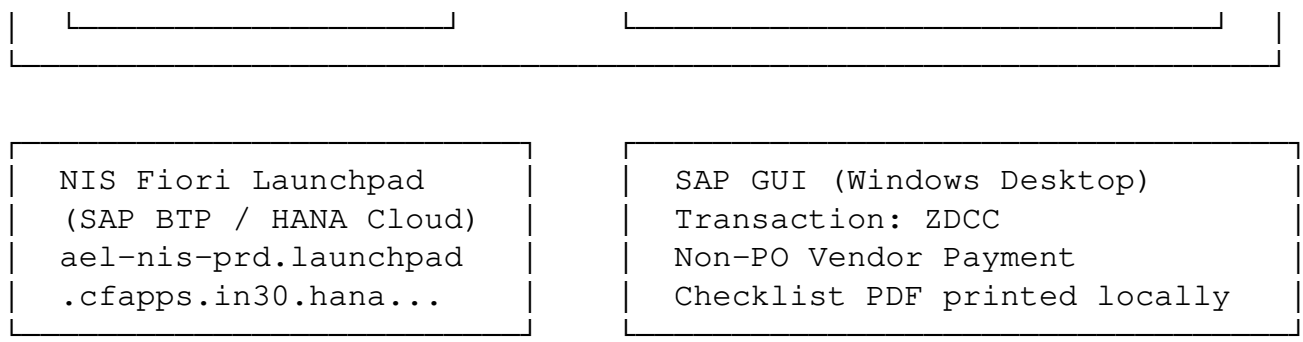
incomplete step.

Architecture



External Systems





Project Structure

```
start7/
│
├── main.py                          # Orchestrator & entry point
├── config.py                        # ★ All credentials, URLs & setti
│
├── read_email.py                    # Fetch DSM emails via Microsoft G
├── classified_html_table_parser.py  # Parse metadata & SPV tables from
│
├── read_vendor_master_data.py       # Pull vendor row from SharePoint
├── add_new_row_data_nis.py          # Write result row to SharePoint E
│
├── nis_booking.py                   # End-to-end NIS portal automation
├── loginwithpassword.py             # Standalone NIS login script (dev
│
├── sap_automation.py                # SAP GUI automation via COM scrip
├── new_sap_automation.py            # Alternative / updated SAP automa
│
├── sheet_db_manager.py              # Sheet-backed persistence layer (
│
├── requirements.txt                 # Python dependencies
├── README.md                        # This file
└── automation_sheet_db.xlsx         # Sheet DB workbook (auto-created
```

Pipeline Steps

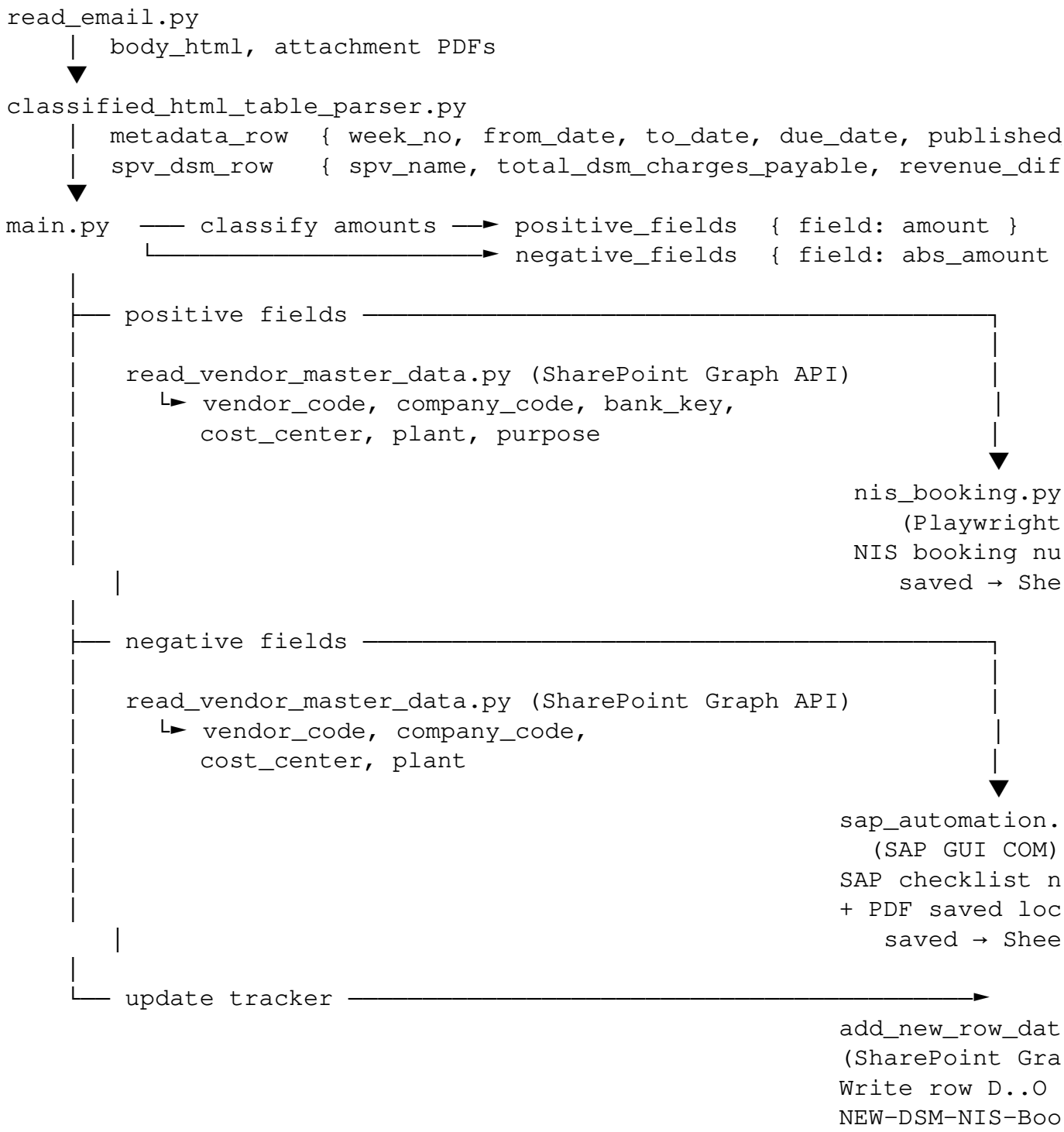
Each step is tracked individually in the Sheet DB with status: `pending → running → done / failed / skipped`.

```
Step 1 | read_email
       | Fetch the DSM email body HTML from Outlook via Graph API.
       | Attachment PDFs are downloaded to ./attachments/{message_id}/
       |
Step 2 | email_extraction
       | Parse two HTML tables from the email body:
       |   • metadata_table  → week_no, from_date, to_date, due_date, p
       |   • spv_dsm_table   → spv_name, total_dsm_charges_payable, dra
       |                       revenue_diff, revenue_loss, net_payable
       |
```

```
Step 3 | email_classification
       | Classify each SPV amount field:
       |   > 0  → positive  → routed to NIS booking
       |   < 0  → negative  → routed to SAP automation (absolute value
       |   = 0  → skipped
       |
Step 4 | email_validation
       | Confirm all required metadata fields are present and at least
       | amount field is actionable. Fails fast with a clear error mess
       |
Step 5 | read_vendor_master
       | Read vendor master row from SharePoint Excel (sheet = spv_name
       |   vendor_code, company_code, cost_center, plant, bank_key, pur
       |
Step 6 | nis_booking_<field>          [one step per positive SPV field]
       | Playwright automation against the NIS Fiori Launchpad:
       |   • Password + MFA login (email → password → RSA OTP → Yes)
       |   • Upload attachment PDF as header document
       |   • Fill Sub Doc Type, Company Code → Process
       |   • Fill Vendor, Bank Key, Invoice Number, Invoice Date
       |   • Add expense item(s): category, sub-category, amount,
       |     cost center, plant, remarks
       |   • Navigate approval screen → select approver radio
       |   • Click Submit
       |   • Capture NIS booking number from success popup
       |   • Click OK to dismiss popup
       |
Step 7 | sap_automation_<field>       [one step per negative SPV field]
       | SAP GUI COM scripting:
       |   • Launch saplogon.exe → attach COM scripting engine
       |   • Login with client / username / password
       |   • Navigate to ZDCC transaction
       |   • Select Non-PO Based → Create checklist
       |   • Fill company code, vendor, year, amount, cost center, plan
       |   • Fill debit note date, doc sub-category, tick documents che
       |   • Preview → Save → confirm popups
       |   • Trigger PDF! print → detect & copy PDF from Temp folder
       |   • Capture checklist number from status bar
       |   • Close all SAP windows
       |
Step 8 | update_excel_tracker
       | Write one new row to SharePoint Excel (columns D-O):
       |   D=Week_No, E=Week range, F=Published date, H=DSM charges,
       |   I=Revenue diff, J=Net payable, L=Due date, M=NIS booking num
```

Data Flow

```
Outlook Email
    |
    |  Graph API (MSAL token)
    ▼
```



Module Reference

File	Role
`main.py`	Entry point. Orchestrates all 8 pipeline steps in order. Supports `--mode scheduled` (poll loop) and `--mode immediate` (single-shot).
`config.py`	Single source of truth for all credentials, URLs, and settings. Every value can be overridden via environment variable.

`read_email.py`	Fetches emails from Outlook via Microsoft Graph API using MSAL. Filters by `DSM-NIS-BOOKING` subject keyword. Downloads PDF attachments.
`classified_html_table_parser.py`	Pure-Python HTML parser. Extracts `metadata_table` and `spv_dsm_table` from the raw email body HTML and returns typed dicts.
`read_vendor_master_data.py`	Reads vendor master data (vendor code, company code, cost center, plant, bank key) from a SharePoint Excel file via Graph API.
`nis_booking.py`	Full Playwright browser automation for the NIS Fiori Launchpad. Handles login (password + RSA MFA), form filling, PDF upload, expense items, approver selection, submit, and NIS number capture.
`loginwithpassword.py`	Standalone script for testing the NIS login flow independently.
`sap_automation.py`	SAP GUI automation via `win32com.client` COM scripting. Fills the ZDCC Non-PO checklist form, saves, prints PDF, and captures the checklist number.
`new_sap_automation.py`	Updated/alternative SAP automation flow with the same public interface as `sap_automation.py`.
`add_new_row_data_nis.py`	Writes one new data row (columns D–O) to the SharePoint Excel tracker sheet via Graph API PATCH.
`sheet_db_manager.py`	Sheet-backed persistence layer. Tracks every email run and every pipeline step with status, timestamps, retry counts, and captured values. Enables full resume on crash.

Configuration

All configuration lives in **`config.py`**. Never hard-code credentials in any other file.

Section	Key Variables
Graph API – Email	`GRAPH_TENANT_ID`, `GRAPH_CLIENT_ID`, `GRAPH_CLIENT_SECRET`, `USER_EMAIL`
Graph API – SharePoint	`SP_TENANT_ID`, `SP_CLIENT_ID`, `SP_CLIENT_SECRET`
SharePoint location	`TENANT_HOST`, `SITE_NAME`, `DRIVE_NAME`, `FOLDER_PATH`, `FILE_NAME`, `TARGET_SHEET`
NIS Portal	`NIS_LOGIN_URL`, `NIS_EDGE_PATH`, `NIS_USERNAME`, `NIS_PASSWORD`, `NIS_OTP_SECRET`
SAP GUI	`SAP_LOGON_PATH`, `SAP_SYSTEM`, `SAP_CLIENT`, `SAP_USERNAME`, `SAP_PASSWORD`, `SAP_LANGUAGE`

SAP Form defaults	`SAP_CHK_DOC_TYP`, `SAP_DIG_SIGN_INV`, `SAP_GJAHR`, `SAP_DEB_NOT_REF`, `SAP_DOC_SUB_CAT`
SAP PDF paths	`SAP_PDF_TEMP_DIR`, `SAP_PDF_TARGET_DIR`
Pipeline	`DSM_SUBJECT_KEYWORD`, `SHEET_DB_PATH`, `ATTACHMENT_DIR`

Environment Variables

Override any config value without touching `config.py`:

```
# Azure AD - Email
export GRAPH_TENANT_ID="your-tenant-id"
export GRAPH_CLIENT_ID="your-app-client-id"
export GRAPH_CLIENT_SECRET="your-secret"
export USER_EMAIL="automation@yourcompany.com"

# Azure AD - SharePoint
export SP_CLIENT_ID="your-sharepoint-app-id"
export SP_CLIENT_SECRET="your-sharepoint-secret"

# NIS Portal
export NIS_USERNAME="nis.user@yourcompany.com"
export NIS_PASSWORD="your-nis-password"
export NIS_OTP_SECRET="your-rsa-otp"
export NIS_EDGE_PATH="C:\Program Files (x86)\Microsoft\Edge\Application

# SAP GUI
export SAP_LOGON_PATH="C:\Program Files (x86)\SAP\FrontEnd\SAPGUI\saplo
export SAP_SYSTEM="Renewable Production SSO"
export SAP_CLIENT="910"
export SAP_USERNAME="AUTO10051"
export SAP_PASSWORD="your-sap-password"

# SAP PDF output
export SAP_PDF_TARGET_DIR="D:\DSM\CheckList-PDF"
```

Installation

Prerequisites

- Python 3.10 or higher
- Windows machine (required for SAP GUI automation via `pywin32` / `pywinauto`)
- Microsoft Edge browser installed at the path set in `NIS_EDGE_PATH`
- SAP GUI (SAP Logon) installed and configured with the system connection

Steps

```
# 1. Clone the repository
git clone https://github.com/Rohitw3code/DSM-NIS-Booking.git
cd DSM-NIS-Booking
```

```
# 2. Create and activate a virtual environment (recommended)
python -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate     # Linux/macOS (non-SAP modules only)

# 3. Install Python dependencies
pip install -r requirements.txt

# 4. Install Playwright browser binaries
playwright install chromium

# 5. Edit config.py with your credentials
#    (or set the corresponding environment variables)
notepad config.py
```

Running the Pipeline

Immediate mode *(run once and exit)*

```
python main.py --mode immediate --lookback 2
```

Scheduled mode *(poll every N seconds)*

```
# Poll every 5 minutes (default), scan last 2 days of email
python main.py --mode scheduled --interval 300 --lookback 2

# Poll every 10 minutes, scan last 1 day
python main.py --mode scheduled --interval 600 --lookback 1
```

Arguments

Argument	Default	Description
`--mode`	required	`immediate` or `scheduled`
`--interval`	`300`	Seconds between polls (scheduled mode only)
`--lookback`	`2`	Number of past days of email to scan

Resume / Fault Tolerance

Every email run and every pipeline step is persisted in `automation_sheet_db.xlsx` (Sheet DB):

```
email_runs table
  id, message_id, subject, status, metadata_json, spv_dsm_json,
  vendor_data_json, nis_checklist_id, checklist_pdf_name, ...

run_steps table
  run_id, step_name, status, attempt_count, detail, error, started_at,
```

Step statuses: `pending → running → done | failed | skipped`

If the pipeline crashes mid-run (e.g. browser crash, network error, SAP timeout):

1. Simply restart with the same command.
2. The pipeline reads `automation_sheet_db.xlsx`, finds the run with status `running` or `failed`.
3. It skips all `done` steps and resumes from the first incomplete step.
4. Each step retries up to **3 times** before being marked permanently `failed`.

SharePoint Excel Sheet Layout

Columns **D → O** written by `add_new_row_data_nis.py`:

Col	Field	Source
D	Week No	`metadata_table.week_no`
E	Week Range	`"from_date to to_date"`
F	DSM Statement Published Date	`metadata_table.dsm_statement_published_date`
G	*(blank)*	—
H	DSM Charges	`spv_dsm_table.total_dsm_charges_payable`
I	Revenue Diff Amount	`spv_dsm_table.revenue_diff`
J	Net Payable / Receivable	`spv_dsm_table.net_payable_receivable`
K	*(blank)*	—
L	Due Date	`metadata_table.due_date`
M	**NIS Booking Number**	Captured from NIS portal success popup
N	*(blank)*	—
O	*(blank)*	—